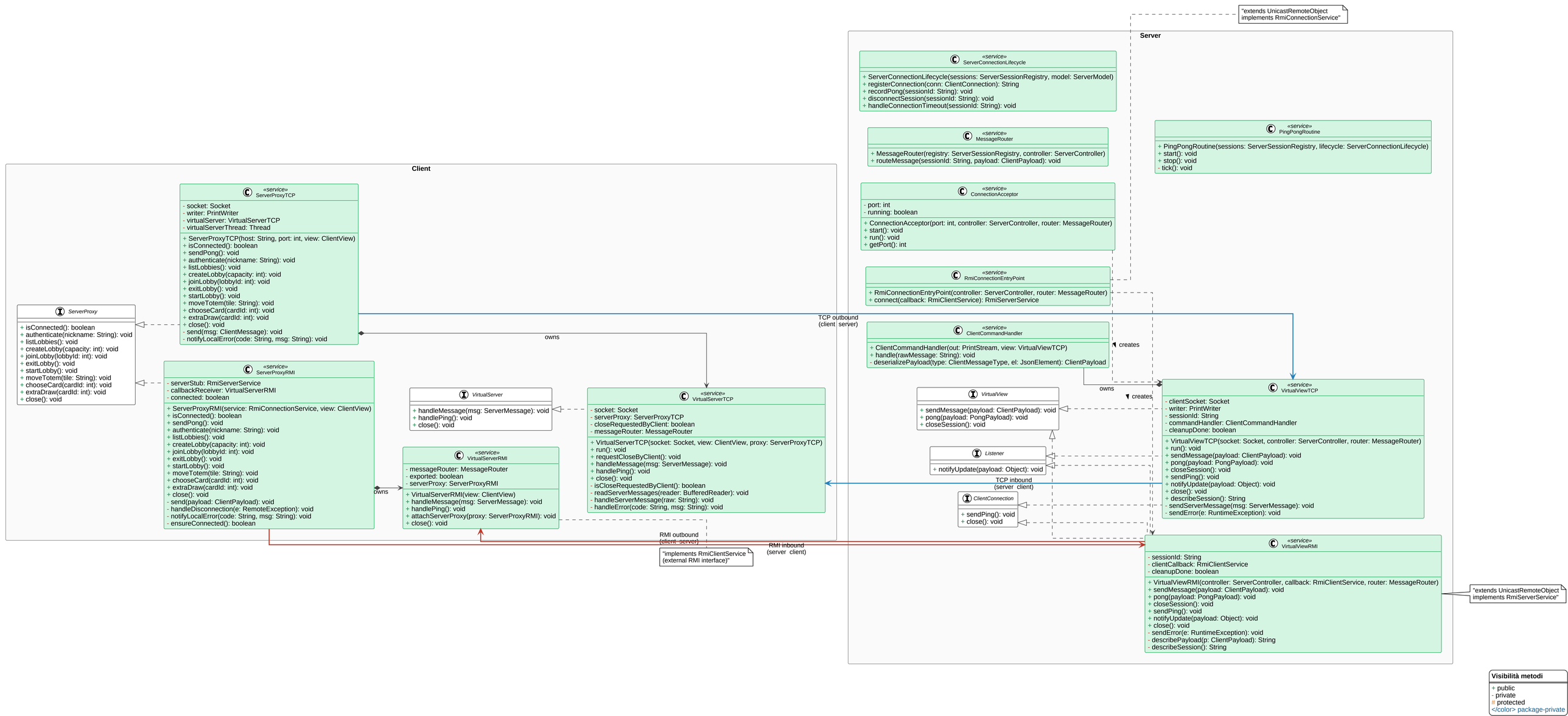
"extends UnicastRemoteObject
implements RmiConnectionService"
Server
«service»
ServerConnectionLifecycle
+ ServerConnectionLifecycle(sessions: ServerSessionRegistry, model: ServerModel)
+ registerConnection(conn: ClientConnection): String
+ recordPong(sessionId: String): void
+ disconnectSession(sessionId: String): void
+ handleConnectionTimeout(sessionId: String): void
«service»
MessageRouter
+ MessageRouter(registry: ServerSessionRegistry, controller: ServerController)
+ routeMessage(sessionId: String, payload: ClientPayload): void
«service»
PingPongRoutine
+ PingPongRoutine(sessions: ServerSessionRegistry, lifecycle: ServerConnectionLifecycle)
+ start(): void
+ stop(): void
- tick(): void
«service»
ConnectionAcceptor
- port: int
- running: boolean
+ ConnectionAcceptor(port: int, controller: ServerController, router: MessageRouter)
+ start(): void
+ run(): void
+ getPort(): int
«service»
RmiConnectionEntryPoint
+ RmiConnectionEntryPoint(controller: ServerController, router: MessageRouter)
+ connect(callback: RmiClientService): RmiServerService
«service»
ClientCommandHandler
+ ClientCommandHandler(out: PrintStream, view: VirtualViewTCP)
+ handle(rawMessage: String): void
- deserializePayload(type: ClientMessageType, el: JsonElement): ClientPayload
creates
owns
creates
VirtualView
+ sendMessage(payload: ClientPayload): void
+ pong(payload: PongPayload): void
+ closeSession(): void
«service»
VirtualViewTCP
- clientSocket: Socket
- writer: PrintWriter
- sessionId: String
- commandHandler: ClientCommandHandler
- cleanupDone: boolean
+ VirtualViewTCP(socket: Socket, controller: ServerController, router: MessageRouter)
+ run(): void
+ sendMessage(payload: ClientPayload): void
+ pong(payload: PongPayload): void
+ closeSession(): void
+ sendPing(): void
+ notifyUpdate(payload: Object): void
+ close(): void
+ describeSession(): String
- sendServerMessage(msg: ServerMessage): void
- sendError(e: RuntimeException): void
Listener
+ notifyUpdate(payload: Object): void
TCP inbound
(server client)
ClientConnection
+ sendPing(): void
+ close(): void
«service»
VirtualViewRMI
- sessionId: String
- clientCallback: RmiClientService
- cleanupDone: boolean
+ VirtualViewRMI(controller: ServerController, callback: RmiClientService, router: MessageRouter)
+ sendMessage(payload: ClientPayload): void
+ pong(payload: PongPayload): void
+ closeSession(): void
+ sendPing(): void
+ notifyUpdate(payload: Object): void
+ close(): void
- sendError(e: RuntimeException): void
- describePayload(p: ClientPayload): String
- describeSession(): String
"extends UnicastRemoteObject
implements RmiServerService"
Client
«service»
ServerProxyTCP
- socket: Socket
- writer: PrintWriter
- virtualServer: VirtualServerTCP
- virtualServerThread: Thread
+ ServerProxyTCP(host: String, port: int, view: ClientView)
+ isConnected(): boolean
+ sendPong(): void
+ authenticate(nickname: String): void
+ listLobbies(): void
+ createLobby(capacity: int): void
+ joinLobby(lobbyId: int): void
+ exitLobby(): void
+ startLobby(): void
+ moveTotem(tile: String): void
+ chooseCard(cardId: int): void
+ extraDraw(cardId: int): void
+ close(): void
- send(msg: ClientMessage): void
- notifyLocalError(code: String, msg: String): void
ServerProxy
+ isConnected(): boolean
+ authenticate(nickname: String): void
+ listLobbies(): void
+ createLobby(capacity: int): void
+ joinLobby(lobbyId: int): void
+ exitLobby(): void
+ startLobby(): void
+ moveTotem(tile: String): void
+ chooseCard(cardId: int): void
+ extraDraw(cardId: int): void
+ close(): void
TCP outbound
(client server)
owns
«service»
ServerProxyRMI
- serverStub: RmiServerService
- callbackReceiver: VirtualServerRMI
- connected: boolean
+ ServerProxyRMI(service: RmiConnectionService, view: ClientView)
+ isConnected(): boolean
+ sendPong(): void
+ authenticate(nickname: String): void
+ listLobbies(): void
+ createLobby(capacity: int): void
+ joinLobby(lobbyId: int): void
+ exitLobby(): void
+ startLobby(): void
+ moveTotem(tile: String): void
+ chooseCard(cardId: int): void
+ extraDraw(cardId: int): void
+ close(): void
- send(payload: ClientPayload): void
- handleDisconnection(e: RemoteException): void
- notifyLocalError(code: String, msg: String): void
- ensureConnected(): boolean
VirtualServer
+ handleMessage(msg: ServerMessage): void
+ handlePing(): void
+ close(): void
«service»
VirtualServerTCP
- socket: Socket
- serverProxy: ServerProxyTCP
- closeRequestedByClient: boolean
- messageRouter: MessageRouter
+ VirtualServerTCP(socket: Socket, view: ClientView, proxy: ServerProxyTCP)
+ run(): void
+ requestCloseByClient(): void
+ handleMessage(msg: ServerMessage): void
+ handlePing(): void
+ close(): void
- isCloseRequestedByClient(): boolean
- readServerMessages(reader: BufferedReader): void
- handleServerMessage(raw: String): void
- handleError(code: String, msg: String): void
owns
«service»
VirtualServerRMI
- messageRouter: MessageRouter
- exported: boolean
- serverProxy: ServerProxyRMI
+ VirtualServerRMI(view: ClientView)
+ handleMessage(msg: ServerMessage): void
+ handlePing(): void
+ attachServerProxy(proxy: ServerProxyRMI): void
+ close(): void
RMI outbound
(client server)
RMI inbound
(server client)
"implements RmiClientService
(external RMI interface)"
Visibilità metodi
+ public
- private
protected
</color> package-private